Q-EVM: Implementation Update

Group 52, CSD Lab

Since the paper, we have implemented the gas-cost part of the Q-EVM design (Sec. IV-A, Fig. 1) so the on-chain cost is now measured by running it, not projected.

What We Implemented

1. **Off-chain ML-DSA verification (Sec. IV-A, Step 2):** The Rust bundler verifies the ML-DSA signature off-chain (Dilithium2, the ML-DSA-44 parameter set), following Algorithm 1.
2. **Groth16 receipt (Sec. I-C, Insight 2):** For each valid operation, the bundler generates a real Groth16 proof (BN254 curve) recording the verification result. The operation hash is a public input of the proof, which prevents replay as in Algorithm 1.
3. **On-chain verification (Sec. IV-A, Step 4):** A Groth16 verifier contract runs on a local EVM (revm), and the gas is taken directly from the EVM's execution result.
4. **Native baseline (Sec. I-A, Table II):** the cost of verifying the same signature directly on-chain is estimated for every signature from the actual ML-DSA verification steps, priced using EVM opcode costs.
5. **Batching (Sec. V-F, Table IV):** One Groth16 proof covers a whole batch, so the on-chain gas is shared across operations.
6. **Dashboard and CLI:** A web dashboard and a command-line demo show these results live for each operation.

Results

Matrix	Paper (Table II)	Our Implementation
ML-DSA verified on-chain	2,840,000 gas	~2,800,000 gas
Q-EVM Groth16 receipt	242,500 gas	~240,400 gas
Gas reduction	91.4%	91.4%

Future Work

1. **ML-DSA inside a real zkVM:** Run the complete ML-DSA verification inside RISC Zero or SP1, so the proof covers the signature check itself.
2. **Custom ML-DSA circuit :** Build a purpose-built arithmetic circuit for ML-DSA to bring proving time from ~12.4 s to under 1 s.
3. **Recursive proof aggregation:** Combine batch proofs recursively to reduce on-chain verification to about 20,000 gas per batch.